How TD-MPC's `(mu, sigma)` update is constructed

This note explains the three lines in `tdmpc_planning.py` that refit the sampling Gaussian each planning iteration:

```
mu    = np.einsum("n,nhd->hd", weights, elite_actions)
var   = np.einsum("n,nhd->hd", weights, (elite_actions - mu) ** 2)
sigma = np.sqrt(var)
```

together with the two lines just above them that build `weights` (`tdmpc_planning.py`):

```
beta    = elite_returns.max()
weights = np.exp((elite_returns - beta) / lam)   # lam = temperature
weights /= weights.sum()
```

Source: Hansen, Wang & Su, *"Temporal Difference Learning for Model Predictive Control"*, arXiv:2203.04955 (TD-MPC), Algorithm 1 (planning / inference).

1. Setup and notation

At one refinement iteration the planner holds a **diagonal Gaussian** over H-step action sequences,

$$q(a) = \mathcal{N}\big(a;\ \mu,\ \mathrm{diag}(\sigma^2)\big), \qquad \mu, \sigma \in \mathbb{R}^{H \times a_{\dim}},$$

i.e. each time step `h` and action coordinate `d` has its own independent mean `mu[h,d]` and std `sigma[h,d]` (no cross-covariance).

It draws candidate sequences, scores each by its planned **return**

$$J(a) \ = \ \sum_{h=0}^{H-1} \gamma^h \, r_h \ + \ \gamma^H V(s_H),$$

(the `_rollout_returns` value — higher is better), then keeps the **top-K elites**. The three lines above turn those K elites back into an updated `(mu, sigma)`.

Array shapes used by the `einsum`:

symbol	code	shape	indices
weights w_n	`weights`	`(K,)`	`n` = elite index
elite actions a_n	`elite_actions`	`(K, H, adim)`	`n, h, d`

The contraction `"n,nhd->hd"` means: **for each `(h,d)`, sum over the K elites**

$$\mu_{h,d} \;=\; \sum_{n=1}^{K} w_n \, a_{n,h,d},$$

i.e. it contracts the elite axis `n` and keeps `h,d`. It is exactly a weighted average of the elite action sequences, computed coordinate-wise.

2. Where the weights come from (MPPI / information-theoretic view)

The weights are **not** ad hoc; they are the optimal reweighting of samples under a KL-regularized control objective — the MPPI / path-integral result that TD-MPC inherits.

Consider choosing a new action distribution `p` that maximizes expected return but stays close (in KL) to a reference `q`:

$$\max_{p} \;\; \mathbb{E}_{a\sim p}\big[J(a)\big] \;-\; \lambda\,\mathrm{KL}\big(p \,\|\, q\big).$$

The calculus-of-variations solution is the **exponential-tilting** (Gibbs) distribution

$$p^{\star}(a) \;\propto\; q(a)\,\exp\!\Big(\tfrac{1}{\lambda} J(a)\Big).$$

We cannot represent `p*` in closed form, but we already have samples $a_n \sim q$. Self-normalized importance sampling against `p*` assigns each sample the weight

$$w_n \;=\; \frac{\exp\big(J(a_n)/\lambda\big)}{\sum_m \exp\big(J(a_m)/\lambda\big)},$$

which is precisely `weights = exp(elite_returns/lam); weights /= weights.sum()`. `lambda` (`temperature`) controls greediness: `lambda -> 0` puts all mass on the single best elite (arg-max), large `lambda` flattens toward a uniform average.

Numerical stability. Subtracting `beta = elite_returns.max()` before the `exp` (line 259) rescales every weight by the same constant $e^{-\beta/\lambda}$, which cancels in the normalization — so it changes nothing mathematically but prevents `exp` overflow. This is the standard log-sum-exp shift.

3. Where the mean/variance formulas come from (weighted MLE view)

Given the weighted samples $\{(a_n, w_n)\}$, TD-MPC fits a fresh diagonal Gaussian to them by **weighted maximum likelihood** (equivalently: minimize the weighted cross-entropy / moment-match `p*`). Maximize

$$\mathcal{L}(\mu,\sigma) \;=\; \sum_{n=1}^{K} w_n \,\log \mathcal{N}\big(a_n;\;\mu,\;\mathrm{diag}(\sigma^2)\big), \qquad \sum_n w_n = 1.$$

Because the Gaussian is diagonal, this separates over each coordinate `(h,d)`. Dropping the indices, for one scalar coordinate with samples x_n:

$$\mathcal{L} = \sum_n w_n \Big[-\tfrac{1}{2}\log(2\pi\sigma^2) - \frac{(x_n-\mu)^2}{2\sigma^2}\Big].$$

Mean. Stationarity in μ:

$$\frac{\partial \mathcal{L}}{\partial \mu} = \sum_n w_n \frac{x_n-\mu}{\sigma^2} = 0 \;\Longrightarrow\; \mu^\star = \frac{\sum_n w_n x_n}{\sum_n w_n} = \sum_n w_n x_n.$$

(The last equality uses $\sum_n w_n = 1$.) That is line 263.

Variance. Stationarity in σ^2:

$$\frac{\partial \mathcal{L}}{\partial \sigma^2} = \sum_n w_n \Big[-\frac{1}{2\sigma^2} + \frac{(x_n-\mu)^2}{2\sigma^4}\Big] = 0 \;\Longrightarrow\; (\sigma^2)^\star = \frac{\sum_n w_n (x_n-\mu)^2}{\sum_n w_n} = \sum_n w_n (x_n-\mu)^2.$$

That is line 264 (`var`), and line 265 takes the square root to recover the std.

So the three lines are the **exact closed-form weighted MLE** of a diagonal Gaussian:

$$\mu_{h,d} = \sum_n w_n \, a_{n,h,d}, \qquad \sigma_{h,d} = \sqrt{\sum_n w_n \, (a_{n,h,d} - \mu_{h,d})^2}.$$

Two implementation details that fall straight out of this:

- **Uses the just-updated `mu` .** The variance line references the `mu` computed one line earlier — the MLE variance is measured about the fitted mean, not the old one.
- **No Bessel ($K-1$) correction.** The MLE divides by the (weighted) sample count, not $K-1$; that is why the formula is $\sum_n w_n(\cdot)^2$ with the normalized weights and nothing else.

4. How this unifies CEM and MPPI

This single update is deliberately a **hybrid** of the two planners in `phase2.py` :

planner	which samples	weighting	updates
CEM	top-K elites (hard)	**uniform** $w_n = 1/K$	mean **and** std
MPPI	**all** N samples	softmax `exp(J/λ)`	mean only
TD-MPC (here)	top-K elites (hard)	softmax `exp(J/λ)` over elites	mean **and** std

- Set the weights uniform ($w_n = 1/K$) and the formulas collapse to `elites.mean(axis=0)` / `elites.std(axis=0)` — **exactly** vanilla CEM (`phase2.py`). TD-MPC's rule is CEM's moment-matching with the uniform average replaced by a return-weighted one.
- Keep the softmax weights but drop the elite truncation and the variance update, and you are back to MPPI's mean-only reward-weighted average (`phase2.py`).

TD-MPC therefore keeps CEM's hard elite truncation (robust to bad tails) **and** MPPI's soft return-weighting (uses the *magnitude* of each elite's advantage,

not just its rank), and refits the full `(mu, sigma)` so the search width adapts each iteration.

5. One caveat this repo exploits

Because the variance is refit by MLE with nothing stopping it, `sigma` can shrink toward zero once the elites cluster — the sampler can **prematurely collapse**. The original TD-MPC floors it (`sigma = max(sigma, eps)`); this repo **removed that floor on purpose** (see the module docstring and `tdmpc_planning.py`) so the collapse can happen and be measured by `_check_premature` (`tdmpc_planning.py`).